

## VAE、Diffusion、Flow Matching 系统讲解（三）Flow Matching 流匹配



冰锐

吉大车辆工程

关注他

1 人赞同了该文章

**【系列文章导航】** VAE、Diffusion 与 Flow Matching 系统性深度讲解

- （一）VAE 变分自编码器
- （二）Diffusion 扩散模型完全指南
- （三）Flow Matching 流匹配（本篇）
- （四）三者对比、Stable Diffusion/DiT 与进阶话题

前两篇我们系统讲解了 VAE 的变分下界、以及 **Diffusion** 全文（前向/反向去噪、预测目标、DDPM/DDIM 与 Score Matching）。在那一套框架里，噪声调度、弯曲的概率路径和较多采样步数，往往是实践中的痛点。本篇进入 **Flow Matching（流匹配）**：从「学一个速度场、用 ODE 把噪声搬到数据」的视角重新组织生成建模，并给出与 Diffusion 的对照、条件流匹配（CFM）核心定理、路径选择与 OT-CFM，最后配有 PyTorch 实现骨架与小结。

### 3. Flow Matching（流匹配）

#### 3.1 核心思想

**从 Diffusion 到 Flow Matching 的动机：**Diffusion 模型效果很好，但存在两个根本性的“遗留问题”：

1. 噪声调度  $\beta_t$  需要精心设计：DDPM 的前向过程  $x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$  中， $\alpha_t$  的变化曲线（linear、cosine 等）对生成质量影响很大，但没有明确的理论指导该如何选择——全靠经验调参。
2. 弯曲的概率路径导致采样慢： $\sqrt{\alpha_t}$  和  $\sqrt{1 - \alpha_t}$  都是  $t$  的非线性函数，所以从噪声到数据的路径是弯曲的。用 ODE solver（如 Euler 方法）积分时，弯曲路径的一阶近似误差大，需要更多步才能准确跟踪。

**Flow Matching 的核心直觉：**既然弯曲是问题的根源，那就**直接走直线**。从噪声到数据的最简单路径就是线性插值，它天然地消除了弯曲带来的所有问题。

**本章逻辑路线图**（建议先浏览全貌，再逐节精读）：

- 3.1-3.2 核心思想和数学基础：学一个速度场驱动 ODE，把噪声“搬运”成数据
- 3.3 为什么不能直接匹配边际速度场？← 和 Diffusion 2.3.2 同样的困难 → 边际速度  $u_t(x)$  依赖整个数据分布，算不出来
- 3.4 Conditional Flow Matching<sup>+</sup> ← 核心突破，本章最重要的部分 → “给定  $(x_0, x_1)$ ”后速度变成常数  $x_1 - x_0$ （和 Diffusion “给定  $x_0$ ”后可算是同一思路）→ CFM 和 FM 梯度相同的证明（L2 损失<sup>+</sup>自动学到平均）
- 3.5 与 Diffusion 的数学联系：Diffusion 是弯曲路径的 Flow Matching
- 3.6-3.7 路径选择与采样：线性/OT 路径 + Euler/Midpoint solver
- 3.8 OT-CFM：用最优传输配对消除路径交叉

Flow Matching 的核心图景——速度场驱动粒子从噪声流向数据：

已赞同 1

添加评论

★ 6

♥ 喜欢

➔ 分享

📄 申请转载

✦ ...

#### 关于作者



冰锐

吉大车辆工程

回答

7

文章

48

关注者

3,597

关注他

发私信

#### 大家都在搜

换一换

美国白宫记者晚宴发生枪... 367 万 新男子性情大变确诊神经梅... 366 万 热

DeepSeek V4 预览版上线... 333 万

AC 米兰国米球员被曝集体... 310 万

华谊兄弟被正式申请破产 297 万

爱喝无糖饮料的天塌了 297 万

东方甄选4名主播集体离职 283 万 新

OpenAI 发布 GPT-5.5 270 万

男童疑被误诊为脑瘫治疗 7... 258 万

张军 253 万

ArkClaw，一键部署 OpenClaw，零门槛即刻唤醒个人助手



ArkClaw  
首月低至50元

一键部署 OpenClaw; doubao、kimi、glm 等  
多模型自由切换

立即购买

广告

[高斯噪声团]

[逐渐成形]

[真实数据点]

每个  $\cdot$  按照速度场  $v_\theta(x, t)$  移动，沿直线路径从噪声位置走到数据位置

**形式化：**学一个**向量场**（速度场） $v_\theta(x, t)$ ，定义连续的 ODE：

$$\frac{dx}{dt} = v_\theta(x, t), \quad t \in [0, 1]$$

- $t=0$ ：噪声  $x_0 \sim \mathcal{N}(0, I)$
- $t=1$ ：数据  $x_1 \sim p_{\text{data}}$

**[注意] 记号变化：**Flow Matching 中  $t=0$  是噪声端、 $t=1$  是数据端，与 Diffusion 的  $t=0$  数据端、 $t=T$  噪声端**方向相反**。后文中  $x_0$  表示噪声样本， $x_1$  表示数据样本。

|      | Diffusion 记号                               | Flow Matching 记号                           |
|------|--------------------------------------------|--------------------------------------------|
| 数据   | $x_0$                                      | $x_1$                                      |
| 噪声   | $x_T$ （或 $\epsilon$ ）                      | $x_0$                                      |
| 时间方向 | $t: 0 \rightarrow T$ （数据 $\rightarrow$ 噪声） | $t: 0 \rightarrow 1$ （噪声 $\rightarrow$ 数据） |

后续公式中遇到  $x_0, x_1$  时，均指 **FM 记号**（ $x_0$ =噪声， $x_1$ =数据），除非特别标注“Diffusion 记号”。

训练好  $v_\theta$  后，从噪声  $x_0 \sim \mathcal{N}(0, I)$  出发，沿 ODE 积分到  $t=1$ ，得到的  $x_1$  就是生成的数据。

**与 Diffusion 的本质区别：**Diffusion 先设计好前向/反向过程（SDE），然后从中推导出训练目标；Flow Matching 直接从“学一个 ODE 的速度场”出发，跳过了 SDE 的所有复杂机制（马尔可夫链、反向后验、变分下界）。整个框架从设计到训练都更直接。

### 3.2 连续正规化流（CNF）的数学基础

**本节的作用：**在定义 Flow Matching 的训练方法之前，先建立必要的数学工具——流映射和**连续性方程**<sup>+</sup>。这些概念来自常微分方程和流体力学，但在这里的用法很直观。

#### 3.2.1 流映射

给定一个向量场  $v_t(x)$ （在每个位置  $x$  和时间  $t$  给出一个速度向量），我们可以定义**流映射**  $\phi_t: \mathbb{R}^d \rightarrow \mathbb{R}^d$ ：

$$\frac{d\phi_t(x)}{dt} = v_t(\phi_t(x)), \quad \phi_0(x) = x$$

**直觉：**把每个点  $x$  想象成一个粒子。 $\phi_t(x)$  是这个粒子在时间  $t$  的位置。在每个时刻，粒子按照当前位置处的速度  $v_t$  移动。 $\phi_0(x) = x$  表示初始时刻粒子在原位。

用一个具体例子：如果  $v_t(x) = c$ （常向量），那么  $\phi_t(x) = x + ct$ ——粒子匀速直线运动。如果  $v_t$  随位置变化，粒子的轨迹就会弯曲。

**关键性质：**在温和条件下，流映射是**一一映射（双射）**——不同的起点一定到达不同的终点，不同的粒子永远不会“撞到一起”。

**为什么“不撞到一起”很重要？**生成模型需要从噪声分布（ $p_0$ ，高斯）映射到数据分布（ $p_1$ ，真实图像）。如果两个不同的噪声点  $x_A$  和  $x_B$  在流动过程中撞到了同一个位置，那从这个位置开始就无法区分它们——信息丢失了，映射不可逆。不可逆意味着：(1) 无法从图像反推出对应的噪声点（信息丢失）；(2) 噪声分布无法拟合真实数据分布。

突比：想家一条河流。

- **Lipschitz 连续（平滑流动）**：河水平缓流动，相邻的两片树叶会沿着相近的路径漂流，永远不会碰到一起——因为一片树叶要追上另一片，必须经过中间的水，而中间的水速度介于两者之间，不允许“穿越”
- **非 Lipschitz（速度突变）**：如果某处水速突然从极慢跳变到极快（像瀑布边缘），相邻的树叶可能一片被冲走、一片留在原地，然后远处的另一片树叶可能被冲到同一个位置——“轨迹”交叉”了

数学上，**Picard-Lindelöf 定理**<sup>\*</sup>保证了：只要  $v_t(x)$  关于  $x$  是 Lipschitz 连续的，ODE 的解存在且唯一——不同初始值的轨迹永远不会交叉。这就是 CNF 可逆的数学保证。

### 3.2.2 连续性方程

如果大量粒子同时按照  $v_t$  流动，它们的概率密度  $p_t(x)$  的演化满足**连续性方程**：

$$\frac{\partial p_t(x)}{\partial t} + \nabla \cdot (p_t(x) v_t(x)) = 0$$

**先用一维的例子理解这个方程。**

想象一条单行道公路（一维）， $p_t(x)$  是位置  $x$  处的车辆密度， $v_t(x)$  是该处的车速。  
考虑位置  $x = 5$  这个点：

- 如果  $x=4$  处的车速慢， $x=6$  处的车速快——车进来得多、出去得多，但出去的更快 → 这个位置的车变少了 → **密度下降**
- 如果  $x=4$  处的车速快， $x=6$  处的车速慢——车快速涌入但出去得慢 → 这个位置堵车了 → **密度上升**

连续性方程就是在描述这件事：**某个位置的密度变化 = 流入的量 - 流出的量。**

**$\nabla$  和  $\nabla \cdot$  是什么？**  $\nabla$ （读作 nabra）本身是一个“偏导数向量”： $\nabla = (\frac{\partial}{\partial x_1}, \frac{\partial}{\partial x_2}, \dots)$ 。它可以和后面的东西做两种运算：

| 运算 | 符号               | 输入 → 输出 | 含义               | 在本文中的出现                           |
|----|------------------|---------|------------------|-----------------------------------|
| 梯度 | $\nabla f$       | 标量 → 向量 | 函数变化最快的方向        | score function<br>$\nabla \log p$ |
| 散度 | $\nabla \cdot F$ | 向量 → 标量 | 向量场在某点“发散”还是“汇聚” | 连续性方程<br>$\nabla \cdot (p_t v_t)$ |

注意  $\nabla \cdot$  中间有一个点  $\cdot$ ，表示点积。所以  $\nabla \cdot \text{vec}\{F\} = \frac{\partial F_1}{\partial x_1} + \frac{\partial F_2}{\partial x_2} + \dots$ 。

**先从一维理解散度**：在一维中  $\nabla \cdot (p v) = \frac{\partial (p v)}{\partial x}$ 。

- $p \cdot v$  可以理解为“流量”——每秒有多少东西经过这个位置（密度 × 速度）
- $\frac{\partial (p v)}{\partial x}$  是流量沿空间的变化率。如果**右边的流量大于左边的流量**，说明东西在从这个位置“流出去” → **散度为正** → 密度下降

**推广到多维**（如二维图像、高维 latent space）： $\nabla \cdot (p_t v_t) = \sum_i \frac{\partial (p_t v_{t,i})}{\partial x_i}$ ，就是把每个维度的“流出量”加起来。直觉不变：**散度为正 = 粒子从该点向外散开；散度为负 = 粒子向该点汇聚。**

**对应到连续性方程**： $\frac{\partial p_t}{\partial t} = -\nabla \cdot (p_t v_t)$ 。散度为正（流出 > 流入）→ 密度下降；散度为负（流入 > 流出）→ 密度上升。这就是**质量守恒**——粒子不会凭空出现或消失，只会从一个地方流到另一个地方。

**核心推论**：给定初始分布  $p_0$ （噪声分布）和向量场  $v_t$ ， $p_t$  在每个时刻都被**唯一确定**。这意味着：**选定一个向量场就完全确定了概率密度的演化轨迹**。如果  $v_t$  选得好， $p_1$  就等于

——以及为什么需要 3.4 节的“条件”版本来解决。

**回溯提示：**这个困难和 Diffusion 2.3.2 节的困难**结构完全相同**。当时是  $q(x_{t-1}|x_t)$  不可算（分母的边际分布要对整个数据分布积分），这里是  $u_t(x)$  不可算（同样要对整个数据分布积分）。如果你已经忘了 2.3.2 的细节，只需记住：**只要出现“对  $p_{\text{data}}$  积分”就不可算，加上“给定某个样本”的条件后就能算**——这个模式在三个模型中反复出现。

### 3.3.1 原始 Flow Matching 目标

我们的目标是：训练一个神经网络  $v_{\theta}(x, t)$  来预测速度场，使得粒子按照这个速度场从噪声分布  $p_0 = \mathcal{N}(0, I)$  流向数据分布  $p_1 = p_{\text{data}}$ 。

最直接的想法：用 L2 损失让  $v_{\theta}$  去拟合“真实的”速度场  $u_t(x)$ ：

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t \sim U[0,1], x \sim p_t} \left[ \|v_{\theta}(x, t) - u_t(x)\|^2 \right]$$

拆解这个损失函数的每一项：

- $t \sim U[0,1]$ ：随机选一个时刻（和 DDPM 的“随机选时间步  $t$ ”一样）
- $x \sim p_t$ ：在  $t$  时刻的概率分布  $p_t$  中采样一个位置  $x$
- $v_{\theta}(x, t)$ ：网络预测“在位置  $x$ 、时刻  $t$ ，粒子应该往哪个方向走、走多快”
- $u_t(x)$ ：**真实的速度场**——如果粒子按照  $u_t$  走， $p_0$  就会恰好变成  $p_1 = p_{\text{data}}$
- $\|v_{\theta} - u_t\|^2$ ：网络预测的速度和真实速度之间的 L2 误差

和 DDPM 损失的对比：

|        | DDPM                                 | Flow Matching            |
|--------|--------------------------------------|--------------------------|
| 网络预测的是 | 噪声 $\epsilon_{\theta}(x_t, t)$       | 速度 $v_{\theta}(x, t)$    |
| 拟合的目标是 | 真实噪声 $\epsilon$                      | 真实速度 $u_t(x)$            |
| 损失函数   | $\ \epsilon - \epsilon_{\theta}\ ^2$ | $\ u_t - v_{\theta}\ ^2$ |

思路完全一样——都是用 L2 损失让网络去拟合一个目标。区别在于预测的物理量不同。

其中  $u_t(x)$  是“真实的”**边际向量场**——满足连续性方程  $\frac{\partial p_t}{\partial t} + \nabla \cdot (p_t u_t) = 0$  的那个速度场。直觉上， $u_t(x)$  回答的问题是：“**如果我想让整个粒子群的密度从  $p_0$ （噪声）平滑地变到  $p_1$ （数据），那么在时刻  $t$ 、位置  $x$  处的粒子应该以什么速度移动？**”

### 3.3.2 为什么这个目标不可用

**问题：** $u_t(x)$  依赖于整个数据分布  $p_{\text{data}}$ ，无法从有限样本中直接计算。

具体来说，要计算  $u_t(x)$ ，你需要知道  $p_t(x)$ ——但  $p_t(x) = \int p_t(x|x_1) p_{\text{data}}(x_1) dx_1$  是对整个数据分布的积分，和 Diffusion 中  $q(x_t) = \int q(x_t|x_0) p_{\text{data}}(x_0) dx_0$  面临的困难完全一样：**你只有样本，没有  $p_{\text{data}}$  的解析形式**。

同样，从  $p_t$  中采样  $x \sim p_t$  也需要知道  $p_t$  的完整形式——但这正是我们要学的东西，陷入了鸡生蛋蛋生鸡的循环。

用一个具体例子感受这个困难：

假设数据分布  $p_{\text{data}}$  是 MNIST 数字图像。在  $t=0.5$  时刻，某个位置  $x$  处有很多粒子——它们有的正在从噪声走向数字“3”，有的正在走向数字“7”，有的走向“9”...

- Diffusion:  $q(x_{t-1}|x_t)$  不可算 (依赖  $q_{\{\text{data}\}}$ )  $\rightarrow$  “给定  $x_0$  ”后变成可算的  $q(x_{t-1}|x_t, x_0)$
- Flow Matching:  $u_t(x)$  不可算 (依赖  $p_{\{\text{data}\}}$ )  $\rightarrow$  “给定  $x_1$  ”后变成可算的  $u_t(x|x_1)$

两个领域面对同样的问题，用了同样的思路：“边际不可算  $\rightarrow$  加条件后可算”。下一节 (3.4) 就是这个突破。

### 3.4 Conditional Flow Matching (核心突破)

本节是 **Flow Matching** 的数学核心，地位等同于 VAE 的 1.3 ELBO 推导和 Diffusion 的 2.3 反向过程推导。它解决了 3.3 节提出的问题：用条件向量场绕过不可算的边际向量场。

**回溯提示：**这里的突破思路和 Diffusion 2.3.3 节完全平行。当时是“给定  $x_0$  后， $q(x_{t-1}|x_t, x_0)$  变成三个已知高斯的贝叶斯运算”；这里是“给定  $(x_0, x_1)$  后，速度变成常数  $x_1 - x_0$ ”。两者都是通过**加条件**把不可算的边际量变成可算的条件量。

#### 3.4.1 关键洞察

**Lipman et al. (2023)** 的核心发现：我们**不需要知道边际向量场**  $u_t(x)$ ！可以用**条件向量场**来等价训练。

思路和 Diffusion 完全平行：

- Diffusion:  $q(x_{t-1}|x_t)$  不可算  $\rightarrow$  “给定  $x_0$  ”后变成可算的  $q(x_{t-1}|x_t, x_0)$
- Flow Matching:  $u_t(x)$  不可算  $\rightarrow$  “给定一对  $(x_0, x_1)$  ”后变成可算的  $u_t(x|x_1)$

给定一对样本：

- $x_0 \sim \mathcal{N}(0, I)$  (噪声)
- $x_1 \sim p_{\{\text{data}\}}$  (数据)

我们可以在这对样本之间定义一条简单的路径，并精确计算沿路径的速度。

#### 3.4.2 定义条件概率路径

最简单的选择——**线性插值**：

$$x_t = (1-t)x_0 + tx_1$$

**为什么选线性插值？**这是从  $x_0$  到  $x_1$  最短、最简单的路径。 $t=0$  时  $x_t = x_0$  (纯噪声)， $t=1$  时  $x_t = x_1$  (纯数据)，中间是两者的加权平均。这条路径是**完全笔直的**——和 Diffusion 的弯曲路径形成鲜明对比。

对应的**条件分布**是（给定数据点  $x_1$ ， $x_t$  的分布）：

$$p_t(x | x_1) = \mathcal{N}(x; tx_1, (1-t)^2 I)$$

推导： $x_t = (1-t)x_0 + tx_1$ ，其中  $x_0 \sim \mathcal{N}(0, I)$ 。给定  $x_1$ ， $x_t$  是  $x_0$  的线性变换，所以仍是高斯分布。均值  $= (1-t) \cdot 0 + t \cdot x_1 = tx_1$ ，方差  $= (1-t)^2 \cdot I$ 。

**验证边界条件：** $t=0$  时  $p_0(x|x_1) = \mathcal{N}(0, I)$  (噪声，不依赖  $x_1$ )； $t=1$  时  $p_1(x|x_1) = \mathcal{N}(x_1, 0) = \delta(x - x_1)$  (退化为数据点本身)。

这就是**条件向量场**  $u_t(x|x_1) = x_1 - x_0$ ——一个**不依赖于  $t$  的常数向量**！

**直觉：**每个粒子从噪声点  $x_0$  出发，以恒定速度  $x_1 - x_0$  沿直线匀速运动到数据点  $x_1$ 。没有加速度、没有弯曲，就是匀速直线运动。

**和 Diffusion 的对比：**Diffusion 的条件向量场（如果把它写成 Flow Matching 的形式）是  $\dot{\sqrt{\bar{\alpha}_t}}x_1 + \sqrt{1-\bar{\alpha}_t}x_0$ ，随  $t$  变化。Flow Matching 的条件向量场是常数  $x_1 - x_0$ ，极度简化。

### 3.4.4 条件 Flow Matching 损失

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_t \left[ \mathbb{E}_{x_0 \sim \mathcal{N}(0, I), x_1 \sim p_{\text{data}}} \left[ \|v_{\theta}(x_t, t) - (x_1 - x_0)\|^2 \right] \right]$$

其中  $x_t = (1-t)x_0 + tx_1$ 。

**训练过程极其简洁**（和 DDPM 对比）：

| 步骤     | Flow Matching (CFM)                      | Diffusion (DDPM)                                                     |
|--------|------------------------------------------|----------------------------------------------------------------------|
| 1. 采样  | 噪声 $x_0 \sim N(0, I)$ ，数据 $x_1$          | 数据 $x_0$ ，噪声 $\epsilon \sim N(0, I)$                                 |
| 2. 选时间 | $t \sim U[0, 1]$                         | $t \sim U\{1, \dots, T\}$                                            |
| 3. 造输入 | $x_t = (1-t)x_0 + tx_1$                  | $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{(1-\bar{\alpha}_t)}\epsilon$ |
| 4. 损失  | $\ v_{\theta}(x_t, t) - (x_1 - x_0)\ ^2$ | $\ \epsilon_{\theta}(x_t, t) - \epsilon\ ^2$                         |

**对比观察：**两者的结构惊人地相似——都是“造一个中间点  $x_t$ ，让网络预测某个目标”。区别在于：（1）插值方式不同（线性 vs  $\sqrt{\bar{\alpha}_t}$  加权）；（2）预测目标不同（速度  $x_1 - x_0$  vs 噪声  $\epsilon$ ）。Flow Matching 的插值和目标都更简单。

### 3.4.5 为什么 CFM 等价于 FM？（核心定理）

#### 先搞清楚所有符号

在进入证明之前，先把所有符号整理清楚：

| 符号                           | 含义     | 具体是什么                               |
|------------------------------|--------|-------------------------------------|
| $x_0$                        | 噪声（起点） | 从标准高斯 $N(0, I)$ 中采样的随机噪声            |
| $x_1$                        | 数据（终点） | 从数据集中采样的真实图像（如一张 MNIST 数字“3”）       |
| $q(x_1)$ 或 $q_{\text{data}}$ | 数据分布   | 所有真实图像的概率分布（我们只有样本，不知道解析形式）         |
| $x_t = (1-t)x_0 + tx_1$      | 中间状态   | $t$ 时刻的插值点， $t=0$ 是纯噪声， $t=1$ 是真实图像 |
| $p_t(x)$                     | 边际分布   | $t$ 时刻所有粒子的密度分布（算不出来）               |
| $p_t(x x_1)$                 | 条件分布   | 给定目标 $x_1$ 后， $t$ 时刻这一个粒子的分布（能算）    |
| $u_t(x)$                     | 边际向量场  | 位置 $x$ 处所有粒子的平均速度（算不出来）             |
|                              |        | 给定目标 $x_1$ 后这个粒子的                   |



知乎

关注

推荐

热榜

专栏

圈子<sup>New</sup>

付费咨询

知学堂

Q

D 直答



消息

私信

**“边际” vs “条件” 的区别**（贯穿整个证明的核心概念）：

用一个具体场景理解。假设一个十字路口有 100 辆车经过：

- **条件速度**  $u_t(x|x_1)$ ：你知道这辆车要去哪（比如去北京），那它的速度和方向是确定的
- **边际速度**  $u_t(x)$ ：你不知道每辆车要去哪，只能看到 100 辆车在路口的平均速度——有的向北、有的向南，平均下来可能速度很慢
- **条件分布**  $p_t(x|x_1)$ ：知道车要去北京，它在  $t$  时刻**这辆车**大概在哪儿（沿着去北京的路线某处）
- **边际分布**  $p_t(x)$ ：不管去哪， $t$  时刻**所有车**在空间中的整体分布

## FM 和 CFM 到底是什么

两个损失函数的含义：

**FM (Flow Matching)** ——理想版，算不出来：

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t, x \sim p_t} [\|v_{\theta}(x, t) - u_t(x)\|^2]$$

含义：在随机时刻  $t$ 、从  $p_t$  中采样位置  $x$ ，让网络预测的速度去拟合**所有粒子的平均速度**  $u_t(x)$ 。问题： $p_t$  和  $u_t$  都算不出来。

**CFM (Conditional Flow Matching)** ——实际版，可以算：

$$\mathcal{L}_{\text{CFM}} = \mathbb{E}_{t, x_1 \sim q, x_0 \sim \mathcal{N}} [\|v_{\theta}(x_t, t) - (x_1 - x_0)\|^2]$$

含义：随机采一对  $(x_0, x_1)$ ，插值得到  $x_t$ ，让网络预测的速度去拟合**这一个粒子的速度**  $x_1 - x_0$ 。一切都能算。

## 核心定理

**定理** (Lipman et al. 2023)： $\nabla_{\theta} \mathcal{L}_{\text{CFM}} = \nabla_{\theta} \mathcal{L}_{\text{FM}}$ 。

虽然两个损失的值不同，但对**网络参数  $\theta$  的梯度完全相同**，所以梯度下降训练出来的网络是同一个。

**先用一个极简的例子理解“值不同但梯度相同”。**

两个函数： $f(\theta) = \theta^2 + 5$  和  $g(\theta) = \theta^2 + 100$ 。值不同 ( $f(3) = 14$ ,  $g(3) = 109$ )，但梯度完全相同 ( $f'(\theta) = g'(\theta) = 2\theta$ )，因为常数项求导后消失了。用梯度下降优化  $f$  和优化  $g$ ， $\theta$  走的路径完全一样。

CFM 和 FM 就是这个关系——差了一个和  $\theta$  无关的常数项。

## 证明（分三步）

**第一步：两种采样方式产生的  $x$  分布相同。**

FM 是直接从  $p_t$  中采样  $x$ 。CFM 是先采  $x_1$ （数据）和  $x_0$ （噪声），再算  $x_t = (1-t)x_0 + tx_1$ 。

这两种方式得到的  $x$  分布是一样的吗？**是的**。这就是全概率公式：

$$p_t(x) = \int p_t(x|x_1) \cdot q(x_1) \, dx_1$$

已赞同 1

添加评论

★ 6

❤ 喜欢

➦ 分享

📄 申请转载

✳

...

第一步：展开  $\|v_\theta - \text{target}\|^2$  和  $\|v_\theta - \text{target}\|^2$ ，逐项比较。

FM 和 CFM 都是  $\|v_\theta - \text{target}\|^2$  的形式。展开后有三项：

$$\begin{aligned} \|v_\theta - \text{target}\|^2 &= \underbrace{\|v_\theta\|^2}_{\text{A}} - 2 \underbrace{\langle v_\theta, \text{target} \rangle}_{\text{B}} + \underbrace{\|\text{target}\|^2}_{\text{C}} \end{aligned}$$

逐项比较（取期望后）：

| 项                                          | FM                       | CFM                                                  | 相同？              |
|--------------------------------------------|--------------------------|------------------------------------------------------|------------------|
| A $E[\ v_\theta\ ^2]$                      | $x$ 来自 $p_t$             | $x_t$ 来自“先采 $(x_0, x_1)$ 再插值”                        | 相同（第一步证明了两种采样等价） |
| B $E[\langle v_\theta, \text{目标} \rangle]$ | 目标是 $u_t(x)$ （平均速度）      | 目标是 $u_t(x x_1)$ （个体速度）；但对 $x_1$ 取期望后，个体速度的平均 = 平均速度 | 相同               |
| C $E[\ \text{目标}\ ^2]$                     | $\ u_t(x)\ ^2$ （平均速度的平方） | $\ x_1 - x_0\ ^2$ （个体速度的平方）                          | 不同！              |

**B 项为什么相同？** 因为  $u_t(x) = \mathbb{E}_{q(x_1|x)}[u_t(x|x_1)]$ ，即边际速度就是条件速度的期望。所以  $\mathbb{E}_x[\langle v_\theta, u_t(x) \rangle] = \mathbb{E}_x[\langle v_\theta, \mathbb{E}_{x_1}[u_t(x|x_1)] \rangle] = \mathbb{E}_{x, x_1}[\langle v_\theta, u_t(x|x_1) \rangle]$ 。

**C 项为什么不同？** 平均值的平方  $\neq$  平方的平均。例如：两个数 3 和 7，平均是 5， $5^2 = 25$ ；但  $3^2 = 9$ ， $7^2 = 49$ ，平均是 29。 $25 \neq 29$ 。

第三步：C 项不含  $\theta$ ，求梯度时消失。

C 项  $\|\text{target}\|^2$  里面：

- FM 的目标  $u_t(x)$ ——这是数据决定的真实速度，和网络参数  $\theta$  无关
- CFM 的目标  $x_1 - x_0$ ——这是数据和噪声决定的，也和  $\theta$  无关

所以对  $\theta$  求梯度时，C 项对  $\theta$  的梯度为 0（C 项与  $\theta$  无关）。梯度只来自 A 和 B，而 A 和 B 在 FM 和 CFM 中完全相同。

结论： $\nabla_\theta \mathcal{L}_{\text{CFM}} = \nabla_\theta \mathcal{L}_{\text{FM}}$ 。

### 直觉理解（最重要）

其实不需要上面的证明也能理解为什么 CFM 有效。核心原因就是 **L2 损失的最优解是条件期望**（2.7.2 节已经证明过）。

想象网络看到了  $(x_t, t)$ ，用 CFM 训练时：

- 第 1 次训练： $x_t$  对应的是  $x_1 = \text{数字“3”}$ ，目标速度 = 6
- 第 2 次训练：同一个  $x_t$  对应的是  $x_1 = \text{数字“7”}$ ，目标速度 = -2
- 第 3 次训练：同一个  $x_t$  对应的是  $x_1 = \text{数字“9”}$ ，目标速度 = 2

L2 损失  $\|v_\theta - \text{target}\|^2$  的最优解是什么？条件期望 =  $(6 + (-2) + 2) / 3 = 2$ 。

这恰好就是**边际速度**  $u_t(x)$ ！所以 CFM 虽然每次只用一个个体的速度训练，但 L2 损失会**自动让网络学到所有个体的平均速度**。

和 DDPM 完全一样的道理：DDPM 每次随机采一个  $\epsilon$ ，网络学到的最优解是  $\mathbb{E}_{\epsilon}[x_t]$ （所有噪声的平均）。CFM 每次随机采一个  $x_1 - x_0$ ，网络学到的



个  $\sqrt{1-\bar{\alpha}_t}$  反推得到  $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$ 。理解 Diffusion 和 Flow Matching 的大义，不仅能够帮助我们对两者的理解，更重要的是能看到 **Diffusion 是 Flow Matching 框架下的一个特例——**选择了弯曲路径的 Flow Matching。

**回溯提示：**本节频繁引用 2.2.2 节的前向闭式解  $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$  和 2.7.3 节的 Probability Flow ODE。核心要点是：Diffusion 的  $\sqrt{\bar{\alpha}_t}$  系数随  $t$  非线性变化（因为  $\bar{\alpha}_t = \prod \alpha_s$  是连乘积再开根号），而 FM 的系数是  $t$  和  $(1-t)$ ——前者弯曲，后者笔直。

### 3.5.1 统一框架：概率路径

两者都可以看作是定义了一条**概率路径**  $p_t(x)$ ，从一个分布流向另一个分布。核心区别在于路径的“形状”：

**概率路径的直觉：**概率路径描述的不是“一个点的运动轨迹”，而是**整个分布（一大堆粒子的集体）从一种形态流向另一种形态的演化过程**。想象一大堆沙子：起点  $t=0$  时堆成标准高斯（圆形、无结构的噪声），终点  $t=1$  时重新排列成真实数据分布的形状。 $p_t(x)$  描述的就是在每个中间时刻  $t$ ，这堆沙子的分布长什么样。之所以叫“路径”，是因为可以想象成在“分布的空间”里走的一条路——从高斯分布走到数据分布。

|      | Diffusion                                                                       | Flow Matching                                    |
|------|---------------------------------------------------------------------------------|--------------------------------------------------|
| 概率路径 | $p_t(x) = \int N(x; \sqrt{\bar{\alpha}_t}x_1, (1-\bar{\alpha}_t)I) q(x_1) dx_1$ | $p_t(x) = \int N(x; tx_1, (1-t)^2I) q(x_1) dx_1$ |
| 条件插值 | $x_t = \sqrt{\bar{\alpha}_t}x_1 + \sqrt{1-\bar{\alpha}_t}x_0$                   | $x_t = (1-t)x_0 + tx_1$                          |
| 路径形状 | 弯曲（ $\sqrt{\bar{\alpha}_t}$ 是 $t$ 的非线性函数）                                       | 直线（系数 $(1-t)$ 和 $t$ 是线性的）                        |
| 学习目标 | 噪声 $\epsilon$ （等价于 score）                                                       | 速度 $v = x_1 - x_0$                               |
| 时间范围 | $t \in \{0, 1, \dots, T\}$ （离散，通常 $T=1000$ ）                                    | $t \in [0, 1]$ （连续）                              |

**公式拆解：**以 Diffusion 的概率路径  $p_t(x) = \int \mathcal{N}(x; \sqrt{\bar{\alpha}_t}x_1, (1-\bar{\alpha}_t)I) q(x_1) dx_1$  为例，这个公式由三部分组成：

- $q(x_1)$ ：真实数据分布。 $x_1$  是某一个真实样本， $q(x_1)$  是它被采到的概率。
- $\mathcal{N}(x; \sqrt{\bar{\alpha}_t}x_1, (1-\bar{\alpha}_t)I)$ ：在时刻  $t$ ，围绕数据点  $x_1$  的“噪声云”。以  $\sqrt{\bar{\alpha}_t}x_1$  为中心、 $(1-\bar{\alpha}_t)$  为方差的高斯——当  $t$  接近终点时噪声云收缩到数据点本身，当  $t$  接近起点时噪声云膨胀成标准高斯。
- $\int \cdots dx_1$ ：对所有可能的真实数据  $x_1$  按出现概率  $q(x_1)$  加权叠加。

**具体例子：**假设数据集只有 3 张图片——猫 A（概率 0.5）、猫 B（概率 0.3）、猫 C（概率 0.2）。在某时刻  $t$ ， $p_t(x)$  就是 3 个高斯的混合分布：猫 A 贡献一个以  $\sqrt{\bar{\alpha}_t}x_{\text{A}}$  为中心的高斯（ $x_{\text{A}}$  表示猫 A 的图像向量，权重 0.5），猫 B 和猫 C 类似。当  $t \rightarrow 0$  时三个高斯都很“胖”且重叠在原点附近（看起来像纯噪声）；当  $t \rightarrow 1$  时三个高斯都很“瘦”且分别集中在三张猫脸的位置（看起来像 3 个尖峰，即真实数据分布）。**这个从“一大坨”变成“三个尖峰”的动态过程，就是概率路径。** Flow Matching 的公式结构完全一样，只是中心从  $\sqrt{\bar{\alpha}_t}x_1$  变为  $t \cdot x_1$ （线性），方差从  $(1-\bar{\alpha}_t)$  变为  $(1-t)^2$ （也与  $t$  线性相关），所以噪声云的收缩过程是匀速的，不像 Diffusion 那样忽快忽慢。

**条件插值公式的来源与直觉：**

表格中的“条件插值”行  $x_t = \sqrt{\bar{\alpha}_t}x_1 + \sqrt{1-\bar{\alpha}_t}x_0$  就是 Diffusion 的前向闭式解（2.2.2 节推导），只是换了 FM 记号。它做的事情非常简单——**按时间  $t$  决定的权重，把数据和噪声混合在一起：**

$$x_t = \underbrace{\sqrt{\bar{\alpha}_t}}_{\text{data}} x_1 + \underbrace{\sqrt{1-\bar{\alpha}_t}}_{\text{noise}} x_0$$

知乎

关注

推荐

热榜

专栏

圈子<sup>New</sup>

付费咨询

知学堂

Q

直答



消息

私信

- $t$  接近起点 ( $\bar{\alpha}_t \rightarrow 0$ ) : 数据权重  $\rightarrow 0$ , 噪声权重  $\rightarrow 1$ ,  $x_t \approx x_0$  (几乎纯噪声)

用  $\sqrt{\cdot}$  而非直接用  $\bar{\alpha}_t$  的原因是保持**方差恒为 1**:  $(\sqrt{\bar{\alpha}_t})^2 + (\sqrt{1-\bar{\alpha}_t})^2 = 1$ , 确保中间状态的分布不会塌缩或发散。

**“弯曲”的来源**:  $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$  本身是连乘积 (非线性), 再开根号后  $\sqrt{\bar{\alpha}_t}$  更加非线性。所以在“数据-噪声”空间里,  $x_t$  随  $t$  画出的轨迹是一条弯曲的曲线。而 Flow Matching 的  $x_t = (1-t)x_0 + t x_1$  中权重就是  $t$  和  $(1-t)$ , 完全线性, 轨迹是一条直线。**这就是“弯曲路径” vs “直线路径”的根本来源。**

**[注意] 记号对应** (重要! 反复查阅此表可避免混淆) :

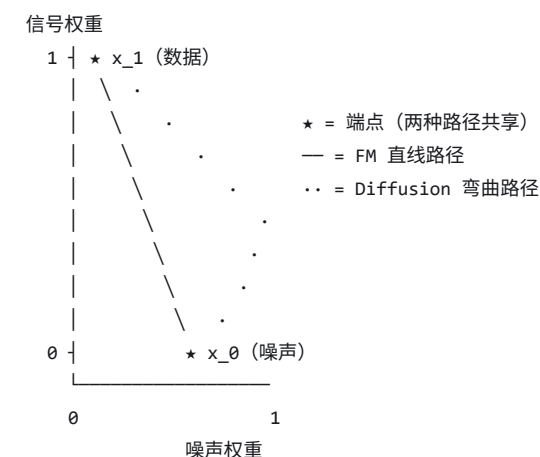
| 上表公式                                                                    | 对应 Diffusion 原始公式 (第 2 章)                                                           | 变量替换                                                                          |
|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| $x_t = \sqrt{\bar{\alpha}_t} x_1 + \sqrt{1-\bar{\alpha}_t} x_0$ (FM 记号) | $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t} \epsilon$ (Diffusion 记号) | FM 的 $x_1$ = Diffusion 的 $x_0$ (数据); FM 的 $x_0$ = Diffusion 的 $\epsilon$ (噪声) |

上表统一用了 FM 记号 ( $x_0$  = 噪声,  $x_1$  = 数据)。

### 3.5.2 路径曲率的影响

为什么路径的“直”与“弯”如此重要?

信号-噪声平面中的概率路径对比 (横轴=噪声权重, 纵轴=信号权重) :



FM 路径:  $x_t = (1-t) \cdot x_0 + t \cdot x_1 \rightarrow$  直线, 速度恒定

Diffusion:  $x_t = \sqrt{\bar{\alpha}_t} \cdot x_1 + \sqrt{1-\bar{\alpha}_t} \cdot x_0 \rightarrow$  弯曲, 速度随  $t$  变化

Euler 积分误差:

直线 ————— (速度不变, 一阶近似精确, 步数少)

弯曲 ~~~~~ (速度变化, 一阶近似有截断误差, 步数多)

当用数值 ODE solver (如 Euler 方法) 采样时, 每一步用的是“一阶近似”:

$$x_{t+\Delta t} \approx x_t + v(x_t, t) \cdot \Delta t$$

这个近似假设速度在  $[t, t+\Delta t]$  区间内不变。实际上:

- **直线路径**: 速度确实不变 ( $v = x_1 - x_0$  是常数), 所以 Euler 方法可以**精确积分**, 理论上 1 步就够 (实际中因为网络预测不完美需要几步)
- **弯曲路径**: 速度随  $t$  变化 (Diffusion 的向量场  $\dot{\sqrt{\bar{\alpha}_t}} x_1 +$

已赞同 1

添加评论

★ 6

♥ 喜欢

➔ 分享

📄 申请转载

✦ ...

### 3.5.3 Diffusion 可以看作特殊的 Flow Matching

如果在 Flow Matching 框架下选择 Diffusion 的插值方式（而非线性插值）：

$$x_t = \sqrt{\bar{\alpha}_t} x_1 + \sqrt{1 - \bar{\alpha}_t} x_0$$

对  $t$  求导，得到条件向量场：

$$u_t(x_1) = \frac{d\sqrt{\bar{\alpha}_t}}{dt} x_1 + \sqrt{1 - \bar{\alpha}_t} \frac{d}{dt} x_0$$

这个向量场是**时间相关的**（不像线性 FM 的  $x_1 - x_0$  是常数），反映了弯曲路径上速度的变化。

用这个向量场做 Flow Matching 训练，得到的模型与 Diffusion 的 Probability Flow ODE 完全等价。所以：

**Diffusion 是 Flow Matching 在弯曲路径下的特例。Flow Matching 通过选择更优的路径（直线），在同一框架内获得了更好的采样效率。**

### 3.6 不同路径选择的比较

Flow Matching 框架的强大之处在于：路径的选择是**自由的**，不同选择对应不同的模型。

| 路径类型             | 插值公式                                                  | 条件向量场                                                        | 特点                                    |
|------------------|-------------------------------------------------------|--------------------------------------------------------------|---------------------------------------|
| 线性（默认）           | $x_t = (1-t)x_0 + tx_1$                               | $x_1 - x_0$ （常数）                                             | 最简单，路径笔直，Euler 方法友好                   |
| 高斯（等价 Diffusion） | $x_t = \sqrt{\alpha_t} x_1 + \sqrt{1 - \alpha_t} x_0$ | $d\sqrt{\alpha_t}/dt x_1 + d\sqrt{1 - \alpha_t}/dt x_0$ （时变） | 路径弯曲，与 DDPM 的 Probability Flow ODE 等价 |
| 最优传输             | 通过 OT 配对 $(x_0, x_1)$ 后做线性插值                          | $x_1 - x_0$ （配对后的常数）                                         | 路径不交叉，向量场最平滑，采样最高效                    |

**选择路径就是选择“从噪声到数据怎么走”**。线性路径最简单但路径可能交叉；OT 路径通过智能配对消除交叉；高斯路径为了兼容 Diffusion 的理论而引入弯曲。实际应用中，线性或 OT 路径是首选。

### 3.7 采样过程

训练好  $v_\theta$  后，采样就是求解 ODE  $\frac{dx}{dt} = v_\theta(x, t)$  从  $t=0$  到  $t=1$ 。

#### 3.7.1 Euler 方法（最简单）

把  $[0, 1]$  均匀分成  $N$  步，步长  $\Delta t = 1/N$ ：

$$x_{t+\Delta t} = x_t + v_\theta(x_t, t) \cdot \Delta t$$

从  $x_0 \sim \mathcal{N}(0, I)$  开始，迭代  $N$  步得到  $x_1$ （生成结果）。

**每步一次网络推理**，所以  $N$  步采样需要  $N$  次前向传播。 $N$  越大越精确但越慢。线性 Flow Matching 通常  $N = 5 \sim 20$  就够了。

#### 3.7.2 高阶 solver（更高效）

↑

已赞同 1

添加评论

★ 6

♥ 喜欢

➔ 分享

📄 申请转载

✦

...

知乎

关注

推荐

热榜

专栏

圈子<sup>New</sup>

付费咨询

知学堂

Q

D 直答



消息

私信

$$\frac{\Delta t}{2} \cdot \Delta t$$

**直觉理解——开车过弯道：**Euler 方法相当于“看一眼当前方向盘角度，闭眼猛踩油门开 100 米”——如果弯道在变，就开偏了。Midpoint 方法是**两步走**：

1. **探路：**用当前速度  $v_{\theta}(x_t, t)$  先走**半步**到中间位置  $x_{\text{mid}} = x_t + v_{\theta}(x_t, t) \cdot \frac{\Delta t}{2}$ ，看看中间地带的速度变成了什么
2. **正式走：****回到起点**  $x_t$ ，用中间位置的速度  $v_{\theta}(x_{\text{mid}}, t + \frac{\Delta t}{2})$  走**完整的一步**

因为中间位置的速度比起点更能代表这一步的“平均速度”，所以走得更准。代码中  $k_1 = \text{model}(x, t)$  是探路速度， $k_2 = \text{model}(x_{\text{mid}}, t_{\text{mid}})$  是中间速度，最终  $x = x + k_2 * dt$  用中间速度走完整步。

先用 Euler 走半步，在半步处重新评估速度，然后用新速度走完整步。**每步需要 2 次网络推理**，但精度是二阶的（每步误差  $O(\Delta t^3)$ ，N 步累积误差  $O(1/N^2)$ ，而 Euler 是  $O(1/N)$ ）——N 步 midpoint  $\approx 2N$  步 Euler 的质量。虽然每步多花一倍计算，但总步数可减半，效果更好。

**RK4**（四阶 Runge-Kutta）：经典的 ODE solver，每步 4 次网络推理，四阶精度。实际中 midpoint 通常已经足够。

**与 DDIM 的对比：**DDIM 本质上也是在求解 ODE（Probability Flow ODE），用的方法和这里类似。Flow Matching 的优势在于 ODE 的“路径更直”，所以用同样阶数的 solver 需要更少步数。

### 3.8 Optimal Transport Flow Matching

#### 3.8.1 路径交叉问题

**问题：**在 CFM 中，每个训练样本随机采一个噪声  $x_0$  和一个数据  $x_1$ ，然后沿直线连接。当有很多样本时，**不同样本的路径可能在中间时刻交叉**。

**具体例子：**假设二维平面上有两对配对：

- 噪声  $x_0^a$ （左边）配了数据  $x_1^a$ （右边）→ 路径向右
- 噪声  $x_0^b$ （右边）配了数据  $x_1^b$ （左边）→ 路径向左

随机配对（路径交叉）：

$x_0^a$  ———— \ ————  $x_1^a$   
 (左)            X        (右)  
 $x_0^b$  ———— / ————  $x_1^b$   
 (右)            (左)

交叉点处速度矛盾！  
 ← 网络被迫输出平均 ( $\approx 0$ ) →

OT 配对（路径不交叉）：

$x_0^a$  ———— ————  $x_1^a$   
 (左)            (左)  
 $x_0^b$  ———— ————  $x_1^b$   
 (右)            (右)

路径平行，无冲突！  
 每个位置速度方向一致

这两条路径会在中间交叉。在交叉点  $x$  处，向量场应该同时指向右（对  $a$ ）和指向左（对  $b$ ）——**网络被迫预测矛盾的方向**，只能输出两者的平均（接近零向量），导致生成质量下降。

**交叉越多，学习越困难：**交叉意味着向量场不平滑，网络需要更高的容量才能拟合，采样也需要更多步数。

**搬家公司类比：**假设有 4 个搬家起点（噪声点）和 4 个目的地（数据点）。如果配对是随机的，可能出现“起点在左边配到右边的目的地，起点在右边配到左边的目的地”这种混乱

已赞同 1

添加评论

★ 6

♥ 喜欢

➔ 分享

📄 申请转载

✈ ...

### 3.8.2 OT-CFM 的解决方案

**思路：**如果我们能**智能地配对** ( $x_0, x_1$ )，让近处的噪声配近处的数据，路径就不会交叉。

在 mini-batch 内用**最优传输** (Optimal Transport) 找到总搬运代价最小的配对：

$$\pi^* = \arg\min_{\pi} \sum_{i,j} \pi_{i,j} \|x_0^{(i)} - x_1^{(j)}\|^2$$

其中  $\pi_{i,j} \in \{0, 1\}$  是配对矩阵，每个  $x_0^{(i)}$  恰好配一个  $x_1^{(j)}$ 。

**直觉：**OT 配对就像“搬家公司调度”——每辆搬家车分配到**离它最近**的目的地，使得所有搬家车的**总路程最短**。这就是经典的**指派问题** (Assignment Problem)，用**匈牙利算法**可求解精确最优解。

**为什么最优配对一定没有交叉？** 如果两条路径交叉了，把交叉的两对**交换目的地** (A 原来去远处、B 原来也去远处  $\rightarrow$  交换后各去近处)，总路程一定变短 (三角不等式保证)。所以总路程最短的方案**不可能存在交叉**——存在交叉说明还能优化，就不是“最优”的。这就像解开一团缠绕的绳子——OT 找到了让所有绳子都不交叉的“最整齐”排列方式。

**效果：**

- 向量场更平滑 (没有矛盾方向)  $\rightarrow$  网络更容易学
- 采样用更少步数就能达到高质量
- 实验表明 OT-CFM 在 FID 等指标上优于普通 CFM

**代价：**每个 mini-batch 需要解一个离散最优传输问题。使用匈牙利算法，复杂度  $O(B^3)$  ( $B$  为 batch size)。对于典型的 batch size (256-512)，开销可接受。

**Mini-batch OT 的局限：**理论上的 OT 应该在**整个数据集和整个噪声分布**之间做全局最优配对，但这在计算上不可行。实际做法只在每个 mini-batch (256-512 个样本) 内做配对，不同 batch 之间的配对可能不一致。但实验表明这个近似已经足够好——每个 batch 内的路径已经不交叉，向量场的平滑性有了显著提升。

**前向引用：**OT-CFM 通过在训练时智能配对来减少路径交叉。还有另一种思路——

**Rectified Flow** (6.2 节) ——在训练后通过迭代 Reflow 来“拉直”已有路径。两者可以结合使用：先用 OT-CFM 训练一个好的初始模型，再用 Rectified Flow 进一步拉直路径，最终实现 1-2 步采样。

## 3.9 特点

| 优势    | 说明                                                               |
|-------|------------------------------------------------------------------|
| 采样快   | 路径直 $\rightarrow$ ODE solver 步数少 (5-20 步 vs Diffusion 的 20-50 步) |
| 训练简单  | 只需一个 L2 回归损失，无需变分下界、SDE 理论                                       |
| 理论优雅  | 基于 CNF + 最优传输，框架统一                                               |
| 无噪声调度 | 不需要设计 $\beta$ schedule，线性插值天然有效                                  |
| 统一性强  | Diffusion 是它的特例 (弯曲路径的 FM)                                       |

| 劣势       | 说明                                          |
|----------|---------------------------------------------|
| OT 配对有限制 | 高维空间中 mini-batch OT 只是近似，大规模时可能不够精确         |
| 生态尚在发展   | 工具链、社区资源不如 Diffusion 丰富 (但在快速追赶)            |
| 条件生成研究较新 | classifier-free guidance 等技术在 FM 上的最佳实践还在探索 |

| 领域   | 代表工作                     | 说明                                 |
|------|--------------------------|------------------------------------|
| 图像生成 | Stable Diffusion 3, Flux | 用 FM 替代了 Diffusion 的 ODE 采样，生成更快更好 |
| 语音合成 | Meta Voicebox            | 非自回归语音生成，速度远快于自回归方法                |
| 文本生成 | Flow Matching for text   | 用连续流替代离散去噪                         |
| 视频生成 | 多个新工作                    | 利用 FM 的快速采样优势处理高维视频数据              |

### 3.11 核心速记卡

看完 Flow Matching 后，闭上文档能说出这些，就算掌握了。

**一句话总结：**直线插值  $x_t = (1-t)x_0 + tx_1$ 、常数速度  $v = x_1 - x_0$ 、不需要噪声调度——这就是 Flow Matching 的全部。

**核心逻辑链（5 步）：**

- 目标：学一个速度场  $v_\theta(x, t)$  驱动 ODE，把噪声分布变成数据分布  
↓
- 直接匹配边际速度场  $u_t(x)$  不可行（依赖整个  $p_{\text{data}}$ ，和 Diffusion 的困难一样）  
↓
- 核心突破：给定一对  $(x_0, x_1)$ ，定义线性路径  $x_t = (1-t)x_0 + tx_1$   
→ 条件速度  $= x_1 - x_0$ （常数！）  
↓
- CFM 损失： $\|v_\theta(x_t, t) - (x_1 - x_0)\|^2$   
→ L2 损失的最优解自动等于边际速度（训练个体、学到平均）  
↓
- 采样：ODE 积分  $x_{t+\Delta t} = x_t + v_\theta \cdot \Delta t$ ，5-20 步即可

**关键公式（3 个）：**

| 公式                                                      | 含义              |
|---------------------------------------------------------|-----------------|
| $x_t = (1-t)x_0 + tx_1$                                 | 线性插值（路径笔直）      |
| $u_t(x x_1) = x_1 - x_0$                                | 条件速度场（常数，不依赖 t） |
| $L_{\text{CFM}} = \ v_\theta(x_t, t) - (x_1 - x_0)\ ^2$ | CFM 损失          |

**和 Diffusion 的核心区别（一句话）：**

- Diffusion：弯曲路径  $\sqrt{1-\alpha_t}x_1 + \sqrt{\alpha_t}x_0 \rightarrow$  需要噪声调度  $\beta_t$ 、需要更多采样步数
- Flow Matching：直线路径  $(1-t)x_0 + tx_1 \rightarrow$  不需要调度、ODE solver 更友好、步数更少

**为什么 CFM = FM？（最核心的直觉）：** L2 损失  $\|v_\theta - \text{target}\|^2$  的最优解是条件期望。每次训练用一个个体的速度  $x_1 - x_0$ ，但 L2 损失自动让网络学到所有个体速度的平均——这就是边际速度场  $u_t(x)$ 。和 Diffusion 中“训练预测单个  $\epsilon$ ，自动学到  $\mathbb{E}[\epsilon|x_t]$ ”完全一样。

### 3.12 PyTorch 实现

已赞同 1

添加评论

★ 6

♥ 喜欢

➦ 分享

📄 申请转载

✈ ...



知乎

关注

推荐

热榜

专栏

圈子<sup>New</sup>

付费咨询

知学堂

Q

D 直答



消息



私信

+ [DIT/MM-DIT](#) 的组成。DIT 的完整架构、与 Flow Matching 的适配优势详见 [5.3](#) 节。

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

# =====
# Part 1: 时间步嵌入 (与 Diffusion 共用同样的正弦编码)
# =====

class SinusoidalTimeEmbedding(nn.Module):
    """
    把连续时间 t in [0, 1] 编码成高维向量。

    与 Diffusion 的时间嵌入 (2.11 节) 完全相同的原理和公式:
        PE(t, 2i) = sin(t / 10000^(2i/d))
        PE(t, 2i+1) = cos(t / 10000^(2i/d))

    唯一区别: Diffusion 的 t 是离散整数 {0, ..., T-1},
    Flow Matching 的 t 是连续浮点数 [0, 1]。
    """
    def __init__(self, dim):
        super().__init__()
        self.dim = dim

    def forward(self, t):
        device = t.device
        half_dim = self.dim // 2
        # log(10000) / (d/2 - 1) 是频率的衰减系数
        emb = math.log(10000) / (half_dim - 1)
        emb = torch.exp(torch.arange(half_dim, device=device) * -emb)
        # t[:, None] * emb[None, :] -> (batch, half_dim)
        emb = t.float()[:, None] * emb[None, :]
        # 拼接 sin 和 cos -> (batch, dim)
        emb = torch.cat([torch.sin(emb), torch.cos(emb)], dim=-1)
        return emb

# =====
# Part 2: 速度场预测网络 v_theta(x_t, t)
# =====

class SimpleVelocityUNet(nn.Module):
    """
    速度场预测网络, 与 Diffusion 的 SimpleUNet (2.11 节) 结构完全相同。

    结构: Encoder(下采样) -> Bottleneck -> Decoder(上采样 + skip connection)
    每一层都注入时间步嵌入, 让网络知道当前所处的时间 t。

    唯一的概念区别:
    - Diffusion 的网络预测噪声 epsilon_theta(x_t, t)
    - Flow Matching 的网络预测速度 v_theta(x_t, t)
    输入输出的维度完全一样 (都是 (B, C, H, W) -> (B, C, H, W)),
    只是训练时的 target 不同。
    """
    def __init__(self, in_channels=1, base_channels=64, time_emb_dim=128):
```

已赞同 1



添加评论



6



喜欢



分享



申请转载



知乎

关注

推荐

热榜

专栏

圈子<sup>New</sup>

付费咨询

知学堂



直答



消息



私信

```

nn.Linear(time_emb_dim, time_emb_dim),
nn.SiLU(),
)

# ----- Encoder (下采样) -----
ch = base_channels
self.enc1 = self._block(in_channels, ch)
self.time_proj1 = nn.Linear(time_emb_dim, ch)
self.pool1 = nn.MaxPool2d(2)

self.enc2 = self._block(ch, ch * 2)
self.time_proj2 = nn.Linear(time_emb_dim, ch * 2)
self.pool2 = nn.MaxPool2d(2)

# ----- Bottleneck -----
self.bottleneck = self._block(ch * 2, ch * 4)
self.time_proj_bn = nn.Linear(time_emb_dim, ch * 4)

# ----- Decoder (上采样 + skip connection) -----
self.up2 = nn.ConvTranspose2d(ch * 4, ch * 2, kernel_size=2, stride=2)
self.dec2 = self._block(ch * 4, ch * 2)
self.time_proj3 = nn.Linear(time_emb_dim, ch * 2)

self.up1 = nn.ConvTranspose2d(ch * 2, ch, kernel_size=2, stride=2)
self.dec1 = self._block(ch * 2, ch)
self.time_proj4 = nn.Linear(time_emb_dim, ch)

self.final = nn.Conv2d(ch, in_channels, kernel_size=1)

def _block(self, in_ch, out_ch):
    """Conv -> GroupNorm -> SiLU -> Conv -> GroupNorm -> SiLU"""
    return nn.Sequential(
        nn.Conv2d(in_ch, out_ch, 3, padding=1),
        nn.GroupNorm(8, out_ch),
        nn.SiLU(),
        nn.Conv2d(out_ch, out_ch, 3, padding=1),
        nn.GroupNorm(8, out_ch),
        nn.SiLU(),
    )

def forward(self, x, t):
    """
    输入:
        x: (B, C, H, W) - 插值后的中间状态 x_t
        t: (B,) - 连续时间 t in [0, 1]
    输出:
        (B, C, H, W) - 预测的速度 v_0(x_t, t)
    """
    # 时间步编码
    t_emb = self.time_mlp(t)

    # Encoder
    e1 = self.enc1(x)
    e1 = e1 + self.time_proj1(t_emb)[:, :, None, None]
    e2 = self.enc2(self.pool1(e1))
    e2 = e2 + self.time_proj2(t_emb)[:, :, None, None]

    # Bottleneck
    b = self.bottleneck(self.pool2(e2))

```

已赞同 1



添加评论

★ 6

♥ 喜欢

➦ 分享

📄 申请转载



```

        e2 = self.up2(e2)
        d2 = self.dec2(torch.cat([d2, e2], dim=1))
        d2 = d2 + self.time_proj3(t_emb)[:, :, None, None]

```

```

        d1 = self.up1(d2)
        d1 = self.dec1(torch.cat([d1, e1], dim=1))
        d1 = d1 + self.time_proj4(t_emb)[:, :, None, None]

```

```

    return self.final(d1)

```

```

# =====
# Part 3: FlowMatching - CFM 的核心算法
# =====

```

```

class FlowMatching:

```

```

    """

```

```

    实现 Conditional Flow Matching 的三个核心操作:

```

1. 线性插值  $x_t = (1-t)x_0 + tx_1$  — 3.4.2 节
2. 训练损失  $\|v_\theta - (x_1 - x_0)\|^2$  — 3.4.4 节
3. ODE 采样 (Euler / Midpoint) — 3.7 节

```

    对比 Diffusion 的 GaussianDiffusion (2.11 节)，最大的区别是：

```

```

    不需要任何预计算的系数表（没有 alpha_bar, beta 等）。

```

```

    """

```

```

    def __init__(self, device='cpu'):

```

```

        self.device = device

```

```

# -----
# 线性插值 - 对应 3.4.2 节
# -----

```

```

    def interpolate(self, x0, x1, t):

```

```

        """

```

```

        x_t = (1-t) * x_0 + t * x_1

```

```

        对比 Diffusion 的 q_sample (前向闭式解) :

```

```

        Diffusion: x_t = sqrt(alpha_bar_t) * x_0 + sqrt(1-alpha_bar_t) * eps

```

```

        FM:         x_t = (1-t) * x_0 + t * x_1

```

```

        FM 的系数是 t 的线性函数 → 路径笔直

```

```

        Diffusion 的系数是 t 的非线性函数 → 路径弯曲

```

```

        """

```

```

        return (1 - t) * x0 + t * x1

```

```

# -----
# 训练损失 - 对应 3.4.4 节
# -----

```

```

    def training_loss(self, model, x1):

```

```

        """

```

```

        CFM 损失:  $L = E[\|v_\theta(x_t, t) - (x_1 - x_0)\|^2]$ 

```

```

        对比 Diffusion 的 training_loss (2.4.4 节简化损失) :

```

| 步骤   | Diffusion                                                      | Flow Matching                      |
|------|----------------------------------------------------------------|------------------------------------|
| 采样   | $\text{eps} \sim N(0, I), t \sim U\{1, \dots, T\}$             | $x_0 \sim N(0, I), t \sim U[0, 1]$ |
| 造输入  | $x_t = \text{sqrt\_ab} * x_0 + \text{sqrt\_1mab} * \text{eps}$ | $x_t = (1-t) * x_0 + t * x_1$      |
| 训练目标 | $\text{loss}(\text{噪声})$                                       | $\text{loss}(\text{位移})$           |

已赞同 1

添加评论

★ 6

♥ 喜欢

➔ 分享

📄 申请转载

✦ ...

```
batch_size = x1.shape[0]
```

```
# 1. 采样噪声 (FM 的  $x_0$  = 起点 = 纯噪声) 和时间
```

```
x0 = torch.randn_like(x1)
```

```
t = torch.rand(batch_size, 1, 1, 1, device=self.device)
```

```
# 2. 线性插值构造  $x_t$ 
```

```
x_t = self.interpolate(x0, x1, t)
```

```
# 3. 条件向量场:  $u_t(x|x_1) = x_1 - x_0$  (常数, 不依赖  $t$ )
```

```
target_velocity = x1 - x0
```

```
# 4. 网络预测速度
```

```
t_flat = t.view(batch_size)
```

```
predicted_velocity = model(x_t, t_flat)
```

```
# 5. MSE 损失
```

```
return F.mse_loss(predicted_velocity, target_velocity)
```

```
# -----
```

```
# Euler 采样 - 对应 3.7.1 节
```

```
# -----
```

```
@torch.no_grad()
```

```
def euler_sample(self, model, shape, num_steps=20):
```

```
    """
```

```
    Euler ODE 采样:  $x_{t+dt} = x_t + v_\theta(x_t, t) * dt$ 
```

```
    从纯噪声  $x_0 \sim N(0, I)$  出发, 沿速度场积分到  $t=1$ 。
```

```
    对比 Diffusion 的 DDPM 采样 (2.6.1 节) :
```

```
- DDPM: 从  $t=T$  到  $t=0$ , 每步用均值公式 + 随机噪声  $z$ 
```

```
- FM Euler: 从  $t=0$  到  $t=1$ , 每步  $x += v*dt$ , 没有随机噪声
```

```
    对比 DDIM 采样 (2.6.2 节) :
```

```
- DDIM 也是确定性 ODE, 但公式更复杂 (先预测  $x_0$  再重新加噪)
```

```
- FM Euler 只需一行:  $x = x + v * dt$ 
```

```
    """
```

```
    dt = 1.0 / num_steps
```

```
    x = torch.randn(shape, device=self.device)
```

```
    for i in range(num_steps):
```

```
        t = torch.full((shape[0],), i * dt, device=self.device)
```

```
        v = model(x, t)
```

```
        x = x + v * dt
```

```
    return x
```

```
# -----
```

```
# Midpoint 采样 - 对应 3.7.2 节
```

```
# -----
```

```
@torch.no_grad()
```

```
def midpoint_sample(self, model, shape, num_steps=10):
```

```
    """
```

```
    Midpoint ODE 采样 (二阶精度) :
```

```
    1. 用当前速度走半步:  $x_{mid} = x_t + v_\theta(x_t, t) * dt/2$ 
```

```
    2. 在中点重新评估速度:  $v_{mid} = v_\theta(x_{mid}, t + dt/2)$ 
```

```
    3. 用中点速度走完整步:  $x_{t+dt} = x_t + v_{mid} * dt$ 
```

已赞同 1



添加评论

★ 6

♥ 喜欢

➦ 分享

📄 申请转载



```

x = x0 / num_steps

```

```

x = torch.randn(shape, device=self.device)

```

```

for i in range(num_steps):

```

```

    t = torch.full((shape[0],), i * dt, device=self.device)

```

```

    t_mid = torch.full((shape[0],), (i + 0.5) * dt, device=self.device)

```

```

    k1 = model(x, t)

```

```

    x_mid = x + k1 * (dt / 2)

```

```

    k2 = model(x_mid, t_mid)

```

```

    x = x + k2 * dt

```

```

return x

```

```

# =====

```

```

# Part 4: OT-CFM - 最优传输配对 (3.8 节)

```

```

# =====

```

```

def ot_cfm_training_loss(model, x1, fm):

```

```

    """

```

```

    在 mini-batch 内用最优传输配对 (x_0, x_1),
    使路径不交叉, 向量场更平滑。

```

```

    和标准 CFM 的唯一区别: 在插值之前, 先用匈牙利算法
    重新配对 x_0 和 x_1, 使总搬运距离最小。

```

```

    """

```

```

    from scipy.optimize import linear_sum_assignment

```

```

    batch_size = x1.shape[0]

```

```

    x0 = torch.randn_like(x1)

```

```

    # 计算代价矩阵:  $C_{ij} = \|x_0^{(i)} - x_1^{(j)}\|^2$ 

```

```

    x0_flat = x0.view(batch_size, -1)

```

```

    x1_flat = x1.view(batch_size, -1)

```

```

    cost = torch.cdist(x0_flat, x1_flat, p=2).pow(2)

```

```

    # 匈牙利算法求最优配对 ( $O(B^3)$ ,  $B=256$  时约几毫秒)

```

```

    row_idx, col_idx = linear_sum_assignment(cost.cpu().numpy())

```

```

    x1_matched = x1[col_idx]

```

```

    # 后续与标准 CFM 完全相同

```

```

    t = torch.rand(batch_size, 1, 1, 1, device=fm.device)

```

```

    x_t = (1 - t) * x0 + t * x1_matched

```

```

    target = x1_matched - x0

```

```

    t_flat = t.view(batch_size)

```

```

    pred = model(x_t, t_flat)

```

```

    return F.mse_loss(pred, target)

```

```

# =====

```

```

# Part 5: 训练循环与使用示例

```

```

# =====

```

```

def train_flow_matching(model, fm, dataloader, optimizer, epochs=100):

```

```

    """

```

```

    标准训练循环 对比 diffusion 的训练 (2.11 节)

```

已赞同 1



添加评论

★ 6

♥ 喜欢

➦ 分享

📄 申请转载



...

```
for epoch in range(epochs):
    total_loss = 0
    for x_batch, _ in dataloader:
        x_batch = x_batch.to(fm.device)

        loss = fm.training_loss(model, x_batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    avg_loss = total_loss / len(dataloader)
    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1}/{epochs}, Loss: {avg_loss:.4f}")

# ===== 完整使用示例（以 MNIST 为例） =====
if __name__ == "__main__":
    from torchvision import datasets, transforms
    from torch.utils.data import DataLoader

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    # 1. 数据: MNIST 28x28 灰度图, 归一化到 [-1, 1]
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5,), (0.5,)),
    ])
    dataset = datasets.MNIST(root="./data", train=True, download=True, transform=
    dataloader = DataLoader(dataset, batch_size=128, shuffle=True)

    # 2. 模型与 Flow Matching
    # 注意: 和 Diffusion 相比, 不需要 GaussianDiffusion 的 beta schedule
    model = SimpleVelocityUNet(in_channels=1, base_channels=64).to(device)
    fm = FlowMatching(device=device)
    optimizer = torch.optim.Adam(model.parameters(), lr=2e-4)

    # 3. 训练
    train_flow_matching(model, fm, dataloader, optimizer, epochs=50)

    # 4. 采样
    model.eval()
    # Euler 采样 (20 步, 简单直接)
    samples_euler = fm.euler_sample(model, shape=(16, 1, 28, 28), num_steps=20)
    # Midpoint 采样 (10 步, 精度更高, 总推理次数相同: 10*2 = 20 次)
    samples_mid = fm.midpoint_sample(model, shape=(16, 1, 28, 28), num_steps=10)

    print(f"Euler samples shape: {samples_euler.shape}")      # (16, 1, 28, 28)
    print(f"Midpoint samples shape: {samples_mid.shape}")     # (16, 1, 28, 28)
```

#### 代码与公式的对应关系:

| 代码                        | 对应的数学公式                                   | 文档章节        |
|---------------------------|-------------------------------------------|-------------|
| SinusoidalTimeEmbedding   | 与 Diffusion 相同的正弦编码, 但输入 $t \in [0,1]$ 连续 | 与 2.11 节相同  |
| $(1 - t) * x_0 + t * x_1$ | $x_t = (1-t)x_0 + tx_1$                   | 3.4.2 线性插值  |
| $x_1 - x_0$               | $u_t(x x_1) = x_1 - x_0$                  | 3.4.3 条件向量场 |

已赞同 1



添加评论

★ 6

♥ 喜欢

➔ 分享

📄 申请转载



...



|                              |                                                                            |                   |
|------------------------------|----------------------------------------------------------------------------|-------------------|
| (Midpoint)                   |                                                                            |                   |
| $x = x + k2 * dt$ (Midpoint) | 用中点速度走完整步                                                                  | 3.7.2 Midpoint 采样 |
| linear_sum_assignment        | $\pi^* = \arg \min \sum \pi_{ij}$<br>$  x_0^{\{(i)\}} - x_1^{\{(j)\}}  ^2$ | 3.8 OT-CFM        |

与 Diffusion 代码的对比（关键差异）：

| 对比项  | Diffusion (2.11 节)                                | Flow Matching (3.12 节)    |
|------|---------------------------------------------------|---------------------------|
| 初始化  | 需要预计算 $\alpha\_bars$ , $\sqrt{\alpha\_bars}$ 等系数表 | 只需 device, 无任何预计算         |
| 造输入  | $\sqrt{ab} * x_0 + \sqrt{1-mab} * \epsilon$       | $(1 - t) * x_0 + t * x_1$ |
| 预测目标 | noise (噪声 $\epsilon$ )                            | $x_1 - x_0$ (位移方向)        |
| 采样公式 | 均值公式 + 随机噪声 (DDPM) 或先估 $x_0$ 再跳 (DDIM)            | $x = x + v * dt$ (一行搞定)   |

本章小结

- **核心简化**：把 Diffusion 的弯曲路径拉直为线性插值  $x_t = (1-t)x_0 + tx_1$ ，速度恒定为  $v = x_1 - x_0$
- **条件技巧**：边际速度场  $u_t(x)$  不可算  $\rightarrow$  给定  $(x_0, x_1)$  后可算  $\rightarrow$  L2 损失自动学到边际平均
- **不需要噪声调度**：没有  $\beta_t$ 、 $\bar{\alpha}_t$  等超参，概率路径直接由线性插值定义
- **与 Diffusion 的联系**：Diffusion 是 Flow Matching 在弯曲路径上的特例 (§3.5)，路径越直  $\rightarrow$  采样步数越少

下一篇预告

（四）三者对比、Stable Diffusion/DiT 与进阶话题 将在同一叙事下对 VAE、Diffusion 与 Flow Matching 做系统对照，并延伸到 Stable Diffusion、DiT 等现代实现与进阶方向，完成本系列的收束与拔高。

直达：（四）三者对比、Stable Diffusion/DiT 与进阶话题

送礼物

还没有人送礼物，鼓励一下作者吧

编辑于 2026-04-25 23:00 · 北京

生成模型



阿里云 × OpenClaw 7\*24小时“AI”助理！

让“AI”干活，解放自己！[查看详情](#)

阿里云 的广告

已赞同 1 添加评论 6 喜欢 分享 申请转载

知乎

关注

推荐

热榜

专栏

圈子<sup>New</sup>

付费咨询

知学堂



直答



消息

私信



理性发言，友善互动

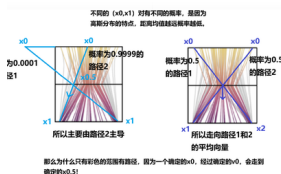
☐ 同时发布到想法

发布



还没有评论，发表第一个评论吧

## 推荐阅读



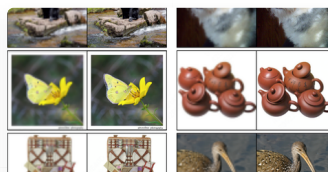
## ShortCut方法：一步的flow matching——附带flow...

肖融ball

## 通俗易懂的Flow Matching原理解读（附核心公式推导和源...

一、前言随着SD3、Flux、MovieGen等系列模型出来以后，flow matching/rectified flow开始逐渐被很多人关注，笔者一开始查了很多帖子，但感觉能通俗易懂的把基本原理讲明白的不多，基本...

byht



## 深入解析Flow Matching技术

笑书神侠

## 系数保持采样：Flow Matching 随机性注入的正确...

阅读本文之前，建议先阅读：零推导理解Diffusion和Flow Matching 和 零推导理解Diffusion Flow Matching（二）：随机性。本文因为涉及到很多推导，没办法作为零推导理解系列的文章了，后...

王峰

已赞同 1

添加评论

★ 6

❤ 喜欢

➔ 分享

📄 申请转载

